# Práctica1. Uso del API de cifrado de Java

**Práctica1. Uso del API de cifrado de Java**

1. Objetivos
2. Descripción
   - 2.1 Simplificaciones
   - 2.2 Requisitos
3. Desarrollo
   - 3.1 Actores
   - 3.2 Programas/módulos a desarrollar
4. Entrega
5. Herramientas a utilizar
   - Código de partida

## 1. Objetivos

- Poner en práctica los conocimientos adquiridos respecto a algoritmos criptográficos
- Conocer y utilizar un API estándar para el desarrollo de aplicaciones criptográficas de complejidad media
  - En este caso se hará uso del API *Java Cryptography Architecture (JCA)* y del provider (implementación de JCA) BouncyCastle.

## 2. Descripción

Se trata de desarrollar una colección de herramientas para el empaquetado y distribución de *Facturas Electrónicas* que sea fiable y segura y garantice las restricciones de entrega en los plazos estipulados por *Hacienda*.

- Para implementar estas restriciciones de entrega se contará con una especie de *Autoridad de sellado de tiempo* simplificada (ver TSA, Autoridad de Sellado de Tiempo)
- Las *Empresas* podrán generar su *Factura Empaquetada* a partir de una de sus *Facturas*. Esta *Factura Empaquetada* será remitida a la *Autoridad de sellado*, que verificará la identidad de la *Empresa* que ha remitido la *Factura Empaquetada* y le vinculará el *timestamp* (sello de tiempo) que permita verificar la fecha de entrega.
- Finalmente, el personal de *Hacienda* podrá validar esta *Factura Empaquetada* para verificar que procede de la *Empresa* correspondiente, extraer la *Factura* original remitida por la *Empresa* y validar la autenticidad del ”sello de tiempo” emitido por la *Autoridad de sellado*.

### 2.1 Simplificaciones

Dado que se trata de una aplicación ”de juguete” se asumirán una serie de simplificaciones.

1. Cada uno de los participantes (*Empresa*, *Autoridad de sellado*, *Hacienda*) podrá generar sus propios pares de claves privada y pública, que se almacenarán en ficheros simples (no se consideran mecanismos adicionales de protección del fichero con la clave privada, como sí ocurriría en una aplicación real)
2. No se contemplan los mecanismos de distribución fiable de claves públicas. Se asumirá que todas las claves públicas necesarias estarán en poder del usuario que las necesite (*Empresa*, *Autoridad de sellado*,

*Hacienda*) de forma confiable (en una aplicación real se haría uso de Certificados Digitales y de una Autoridad Certificadora común)

   - La *Empresa* dispondrá de un fichero con la clave pública de *Hacienda* , utilizada para generar la *Factura Empaquetada*
   - La *Autoridad de sellado* dispondrá del fichero con la clave pública de la *Empresa* que le ha envida su *Factura Empaquetada* para ser sellada
   - *Hacienda* contará con la clave pública (almacenada en su respectivo fichero) de la *Autoridad de sellado*, así como con la clave pública de todas las *Empresas* para las cuales se vaya a realizar la validación de sus *Facturas Empaquetadas*.

3. La *Factura* original de la *Empresa* estará almacenada inicialmente en un fichero JSON (no es relevante para este entregable el formato de la factura, ni será necesario procesarla)
4. Dado que se trata de un ejemplo, las distintas piezas de información que aporte cada participante (*Empresa* o *Autoridad de sellado*) a la *Factura Empaquetada* tendrán (antes del cifrado/firma y después del descifrado) la forma de *Strings* con codificación UTF8.
5. La *Factura Empaquetada* se materializará físicamente en un fichero o "paquete" que contendrá toda las piezas de información que le vayan incorporando los distintos participantes implicados: la *Empresa* que la generó y la *Autoridad de sellado* que da fé de la entrega de la *Factura* en el instante concreto en ésta que tuvo lugar.

   **Nota:**

   - Se aporta código para la gestión de "Paquetes" con múltiples partes codificadas en bloques de caracteres imprimibles empleando codificación Base64 (ver Codificacion BASE64)
   - **NECESARIAMENTE** se **deberá** emplear el código proporcionado o proporcionar una implementación propia equivalente en el caso de desarrollar la prácticas en otros lenguajes diferentes a Java.

6. No se contempla un almacenamiento "físico" realista de la *Factura Empaquetada*, sólo se trata de implementar los programas para generar, sellar y validar la *Factura Empaquetada* conforme a las especificaciones descritas en este documento.
   - En una aplicación real este tipo de datos cifrados y firmados se encapsularían en un formato criptográfico estándar como PKCS7
7. Al validar la *Factura Empaquetada*, si todas las comprobaciones de autenticidad respecto a *Empresa* y *Autoridad de sellado* son correctas, se mostrará al personal de *Hacienda* la *Factura* real remitida por la *Empresa* (el fichero JSON inicial) y los datos (*timestamp*) incorporados por la *Autoridad de sellado* que haya procesado dicho *Factura Empaquetada*. En caso contrario se indicarán las comprobaciones que no hayan sido satisfactorias.

## 2.2 Requisitos

Requisitos básicos a cumplir por el esquema criptográfico propuesto:

- **R1.** Asegurar la **confidencialidad** del contenido incluido en la *Factura Empaquetada* por parte del *Empresa*
  - Sólo el personal de *Hacienda* podrá tener acceso a estos contenidos
- **R2.** Garantizar que tanto el personal de *Hacienda* como cualquier otro participante tenga la posibilidad de verificar que la *Factura Empaquetada* fue realmente presentado por la *Empresa* correspondiente.

- **R3.** **Asegurar** que el contenido del ”paquete” con la *Factura Empaquetada* (datos de la *Empresa* y sello de la *Autoridad de sellado*) que se ha recibido **no** haya sido **modificado** (es decir, que de darse ese caso, se pueda detectar la realización de modificaciones)
- **R4.** Asegurar que ni la *Empresa* ni la *Autoridad de sellado* podrán **repudiar** el contenido incluido por ellos en la *Factura Empaquetada*
- **R5**. **Asegurar** que el personal de *Hacienda* **no** podrá realizar **cambios** en el contenido de la *Factura Empaquetada* recibida (es decir, que de darse ese caso, se pueda detectar la realización de modificaciones)
- **R6.** Contar con un mecanismo mediante el cuál una "Autoridad Externa" confiable por todos los participantes (en este caso, la *Autoridad de sellado de tiempo*) pueda **garantizar la fecha** en que fue presentada la *Factura Empaquetada* generada por una determinada *Empresa*.

  Se pretende que esta vinculación entre *Factura Empaquetada* y su ”sello de tiempo” pueda ser validada por *Hacienda* o por un tercero y que no pueda ser falsificada ni por la *Empresa*, ni por la *Autoridad de sellado*, ni por la propia *Hacienda*

  - **Nota:** En este caso esta *Autoridad de sellado* funcionarán de un modo parecido (aunque simplificado) a las **autoridades de sellado de tiempo** de las infraestructuras de clave pública (ver más en TSA, Autoridad de Sellado de Tiempo).
- **R7.** La *Autoridad de sellado* debe poder verificar que la *Factura Empaquetada* que va a sellar procede realmente de la *Empresa* que la presenta.
- **R8.** Asegurar un **coste computacional reducido** en la creación, sellado y validación de la *Factura Empaquetada* , **minimizando** el uso de **criptografía asimétrica**

# 3. Desarrollo

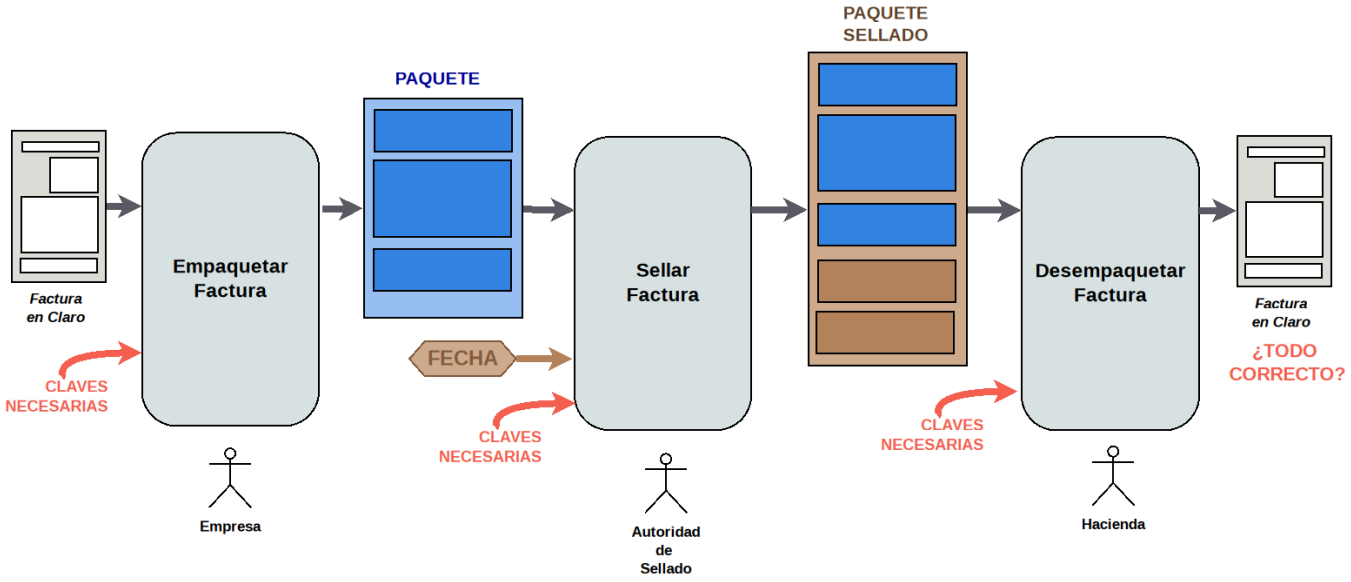
En primer lugar se deberán de analizar los requisitos anteriores, para determinar qué estrategias seguir para conseguir cada uno de ellos.

Se debe decidir qué acciones realizar en el origen (*Empresa*), qué tareas realizará la *Autoridad de sellado* y qué comprobaciones se llevarán a cabo en el destino (*Hacienda*), además de decidir qué algoritmos concretos se emplearán.

## 3.1 Actores

- *Empresas*: podrán generar sus propios pares de claves (pública y privada) y serán las responsables de **generar** la *Factura Empaquetada* a partir del fichero JSON con la *Factura* en claro original.
- *Autoridad de Sellado*: podrá generar su propio par de claves (pública y privada) y será responsable de **sellar** la *Factura Empaquetada* de una *Empresa* dada, habiendo verificado previamente que esa *Factura Empaquetada* efectivamente fue creada por la *Empresa* emisora .
- *Hacienda*: podrá generar su propio par de claves (pública y privada) y será responsable de extraer los datos aportados por la *Empresa* en la *Factura Empaquetada* que le haya enviado, después de haber validado la autenticidad de la autoria de dicha *Factura Empaquetada* y validar la información incluida en la *Factura Empaquetada* por la *Autoridad de sellado*

## 3.2 Programas/módulos a desarrollar




Una vez decidido cómo garantizar los requisitos exigidos, el resultado final será **obligatoriamente** el desarrollo de 4 ejecutables:

- `java -cp [...] GenerarClaves <identificador>` (**ya** está **implementado**)
  - Usado para generar los pares de claves de los participantes: *Empresa*, *Autoridad de sellado* y *Hacienda*
    - Se le pasa como argumento de línea de comando un `identificador` que se usará para componer los nombre de los archivos que se generarán
    - Genera dos ficheros: `"identificador.publica"` e `"identificador.privada"`, conteniendo, respectivamente, las claves pública y privada de ese usuario
- `java -cp [...] EmpaquetarFactura <fichero JSON factura> <nombre paquete> <ficheros con las claves necesarias>`
  - Usado por la *Empresa*
    - Se le pasa en línea de comandos un fichero JSON con el contenido de la *Factura* en claro a empaquetar, el nombre del paquete resultante y el path de los ficheros con las claves necesarias para el empaquetado (el número y tipo exacto de los ficheros de claves dependerá de que estrategia se haya decidido seguir).
    - Genera el fichero `<nombre paquete>` (por ejemplo `factura.paquete`) con el resultado de "empaquetar" los datos de entrada y que conforma la *Factura Empaquetada*.
- `java -cp [...] SellarFactura <nombre paquete> <ficheros con las claves necesarias>`
  - Usado por la *Autoridad de sellado*
    - Se le pasa en línea de comandos el fichero con el "paquete" a sellar y el path de los ficheros con las clave/s criptográficas necesaria/s.
    - Al "paquete" recibido como argumento le vincula (añade) los bloques que correspondan para incorporar los datos aportados por la *Autoridad de sellado* (fecha y hora de entrega) y para garantizar la autenticidad de los datos de "sellado".

        - El resultado será el mismo fichero del ”paquete” pasado como parámetro con los nuevos datos incorporados en forma de nuevos bloques.
        - En caso de comprobar que la *Empresa* que presenta el *Factura Empaquetada* no se corresponde con la que realmente ha creado dicho paquete (no fue creado con la clave privada de la *Empresa* o fue modificado posteriormente), se informará por pantalla y no se generarán los bloques de sellado.
- `java -cp [...] DesempaquetarFactura <nombre paquete> <fichero JSON factura> <ficheros con las claves necesarias>`
    - Usado por los empleados de *Hacienda*
        - Se le pasa en línea de comandos el fichero con el ”paquete” que representa la *Factura Empaquetada* (donde se incluyen los datos [contenido del fichero JSON] aportados por la *Empresa* y los datos de la *Autoridad de sellado*), el nombre del fichero JSON donde se almacenará la *Factura* en claro y el path de los ficheros con las claves que sean necesarias para desempaquetar y verificar la información que contiene el mencionado ”paquete”.
        - Al usuario (*Hacienda*) se le indicará por pantalla el resultado de las diferentes comprobaciones que se hayan realizado sobre la *Factura Empaquetada* y se almacenará un copia en claro de la *Factura* enviada.
            - se indicará si los datos incluidos por la *Empresa* o por la *Autoridad de sellado* han sufrido modificaciones o no
            - se indicará si el ”sello” de la *Autoridad de sellado* es válido/auténtico y, de ser así, se mostrará la fecha de sellado.
            - una vez verificado que la *Empresa* que generó el ”paquete” es quien realmente corresponde, se descifrará la *Factura* enviada y se almacenará en el fichero indicado el texto JSON en claro incluido originalmente por la *Empresa* en su *Factura Empaquetada*

# 4. Entrega

- Práctica **individual** o **en parejas**
- **Documentación a entregar:**
    - Memoria
      Contendrá al menos los siguiente puntos
        1. Descripción breve de la práctica
        2. Descripción y justificación de las estrategias criptográficas empleadas para asegurar los requisitos básicos exigidos, junto con los pasos que se siguen en el empaquetado, sellado y desempaquetado de la *Factura*. **[Importante]**
        3. Descripción del formato/estructura del ”paquete” resultante
        4. Descripción breve de la implementación: clases y métodos más importantes.
        5. Instrucciones de compilación y ejemplos de uso.
           Debe incluir un ejemplo de ejecución (comandos + salida obtenida) de los siguientes **casos de prueba**: **[Importante]**
            - Recepción de una *Factura Empaquetada* correcta
            - Recepción de una *Factura Empaquetada* con los contenidos aportados por la

*Empresa* alterados
        - Recepción de una *Factura Empaquetada* con un sello de tiempo alterado
        - Uso por parte de *Hacienda* de una clave pública de *Empresa* incorrecta
    6. Explicación de las simplificaciones realizadas (las descritas en apartado "2.1 Simplificaciones" y otras adicionales), junto con sus consecuencias en una aplicación real. **[Importante]**
    7. Resultados obtenidos y conclusiones
  - Código fuente + ficheros para los casos de prueba indicados en el punto 5.
    - **Aviso importante:**
      Salvo razones debidamente justificadas en la documentación (uso de otros lenguajes de programación o similares), los ejecutables desarrollados deben de seguir las restricciones respecto a nombre, parámetros de línea de comandos y orden de los mismos especificadas en la sección "3.1 Programas a desarrollar"

    Del mismo modo, los datos aportados por *Empresa* y *Autoridad de sellado* han de al almacenarse en el "Paquete" creado.
- **Fecha de entrega:** por determinar (~ finales de octubre)
- Entrega en MOOVI (un único archivo con documentación, código, etc) [evitar incluir el .JAR de BouncyCastle y ficheros .class]
- **Defensa:** esta previsto que alrededor de un 10%-15% de las prácticas entregadas en tiempo y forma serán seleccionadas para una pequeña defensa (5-10 min.) ante el profesor
  - fundamentalmente serán prácticas con dudas de funcionamiento o implementación, aquellas que presentes dudas en cuanto a su autoría o entregas elegidas al azar
  - la defensa se realizará bien en horario de laboratorio o en horario de tutorias (se acordará con cada grupo)
- **Evaluación:**
  - Documentación correcta + empaquetado básico (*Factura* de la *Empresa*) y validación/descifrado por *Hacienda*: hasta 70%
  - Adición de la generación y validación de los "sellos de tiempo" de la *Autoridad sellado*: hasta 100%

# 5. Herramientas a utilizar

La práctica se implementará en Java utilizando el API de criptografía JCA y el provider Bouncy Castle. Se podrá realizar tanto en Windows como en Linux.

- API Java Cryptography Architecture (JCA) [antes JCE (Java Cryptography Extension)]
- Paquete de criptografia Bouncy Castle (JCA provider)
- Tutorial del API JCA (Java Cryptography Architecture)

## Código de partida

- `GenerarClaves.java`: genera un par de claves RSA de 512 bits
  - Implementación del primer ejecutable descrito en la subsección 3.1 Programas a desarrollar
- `Paquete.java` : encapsula un paquete formado por varios bloques. Cada boque tiene un nombre (de tipo `String`) y un contenido (de tipo `byte[]`).

    - La clase provee de métodos para añadir, listar y eliminar los bloques que conforman el paquete y recuperar su contenido (`byte[]`).

      Métodos a utilizar: `getContenidoBloque()`, `anadirBloque()`, `actualizarBloque()`, `eliminarBloque()`, `getNombresBloque()`
    - También provee métodos para las operaciones de lectura y escritura de paquetes empleando un formato similar al PGP *ASCII armor* (ver ) que usa la codificación Base64 para representar datos binarios mediante caracteres imprimibles.

      Métodos: `leerPaquete(String nombreFichero)`, `escribirPaquete(String nombreFichero)` y constructor de conveniencia `Paquete(String nombreFichero)`
    - En el método `main()` se proporciona un ejemplo de creación, acceso, escritura y lectura de Paquetes.
- `factura2203.json`: Ejemplo de factura en formato JSON